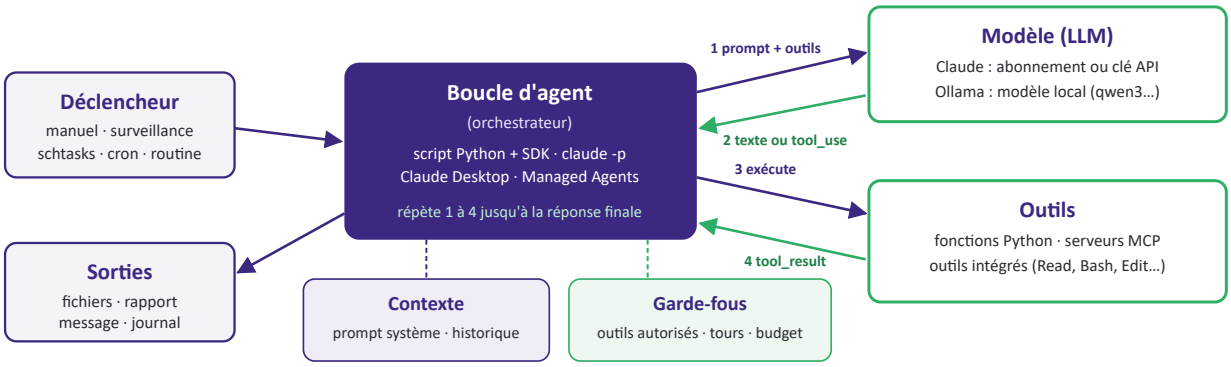
Agent IA : composants et usage en local

Boucle d'agent, modèle, outils · 3 modes : abonnement Claude, clé API, local gratuit (Ollama) · déclencheurs



Composants

Composant	Rôle	Dans le code
Modèle	Lit le contexte, répond en texte ou demande un outil (`tool_use`). N'exécute rien.	`model=`
Boucle d'agent	Envoie la requête, exécute les outils demandés, renvoie les résultats, s'arrête sur réponse finale ou limite.	`query()` , `tool_runner` , boucle manuelle
Outils	Actions sur le monde : fichiers, commandes, API. Décrits par nom, description, schéma JSON.	`@tool` , `@beta_tool` , serveur MCP
Contexte	Prompt système, historique des tours, résultats d'outils, fichiers lus.	`system_prompt=` , `messages=`
Garde-fous	Limiter ce qui s'exécute sans validation, le nombre de tours, la dépense.	`allowed_tools` , `max_turns` , `max_iterations`
Déclencheur	Démarre un run : humain, événement fichier, horaire.	terminal, `surveiller.py` , `schtasks`
Sorties	Résultat exploitable et trace du run.	fichier, `ResultMessage` , log

Un tour de boucle

1. La boucle envoie au modèle : prompt système, historique, définitions des outils.
2. Le modèle répond : texte final, ou un ou plusieurs blocs `tool_use` (nom + arguments JSON).
3. La boucle vérifie l'autorisation, exécute l'outil localement.
4. Le résultat repart au modèle dans un bloc `tool_result` . Retour en 1.

Arrêt : réponse sans `tool_use` , nombre maximal de tours atteint, budget atteint, ou erreur.

Choisir un mode

	Abonnement Claude	Clé API	Local (Ollama)
Auth.	compte Claude (navigateur)	`ANTHROPIC_API_KEY`	aucune
Coût	inclus, consomme le quota	facturé au token	gratuit, CPU/GPU/RAM du poste
Outils	Agent SDK, `claude -p` , Claude Desktop, MCP	SDK `anthropic` , tool runner, Managed Agents	`ollama` Python, API locale
Boucle	fournie (moteur Claude Code)	tool runner, manuelle, ou hébergée (Managed Agents)	à écrire
Limite	quota partagé avec l'usage chat	coût à surveiller	qualité et vitesse selon modèle et matériel

Priorité des identifiants. Si `ANTHROPIC_API_KEY` est définie, le SDK et `claude` l'utilisent à la place de l'abonnement. Pour rester sur l'abonnement : retirer la variable. L'Agent SDK fonctionne aussi avec une clé API.

Mode A · Abonnement Claude

Agent SDK Python

Installation (PowerShell)

```
# 1. Python 3.12 sans droits admin (si échec : installateur python.org)
winget install -e --id Python.Python.3.12 --scope user
python --version
# 2. Dossier et environnement virtuel
mkdir $HOME\agent-sdk; cd $HOME\agent-sdk
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install claude-agent-sdk
# 3. Connexion au compte Claude (ouvre le navigateur)
.\.venv\Lib\site-packages\claude_agent_sdk\_bundled\claude.exe `
    auth login --claudeai
.\.venv\Lib\site-packages\claude_agent_sdk\_bundled\claude.exe `
    auth status                          # "loggedIn": true
# 4. Aucune clé API ne doit prendre le dessus
Remove-Item Env:ANTHROPIC_API_KEY, Env:ANTHROPIC_AUTH_TOKEN `
    -ErrorAction SilentlyContinue
```

Le SDK embarque son propre `claude.exe` . Si Claude Code est déjà installé et connecté (`claude auth status`), l'étape 3 est inutile. Variante sans navigateur : `claude setup-token` puis `$env:CLAUDE_CODE_OAUTH_TOKEN = "<jeton>"` .

Agent minimal avec un outil maison

```
import asyncio
from datetime import datetime
from claude_agent_sdk import (ClaudeAgentOptions, ResultMessage,
    SystemMessage, create_sdk_mcp_server, query, tool)

@tool("heure_actuelle", "Donne la date et l'heure locales.", {})
async def heure_actuelle(args):
    t = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    return {"content": [{"type": "text", "text": t}]}

serveur = create_sdk_mcp_server(name="outils", version="1.0.0",
                                tools=[heure_actuelle])
options = ClaudeAgentOptions(
    model="claude-haiku-5-5",          # modèle explicite
    system_prompt="Tu es un assistant concis.",  # prompt court
    tools=[],                          # aucun outil intégré
    mcp_servers={"outils": serveur},
    allowed_tools=["mcp__outils__heure_actuelle"],
    max_turns=3, setting_sources=[])

async def main():
    async for m in query(prompt="Quelle heure est-il ?",
                         options=options):
        if isinstance(m, SystemMessage) and m.subtype == "init":
            # 'none' = aucune clé API utilisée
            print("auth :", m.data.get("apiKeySource"))
        elif isinstance(m, ResultMessage):
            print(m.result, m.total_cost_usd, m.usage)

asyncio.run(main())
```

Quota. Sans `model=` ni `system_prompt=` , le SDK envoie le prompt système complet de Claude Code avec le modèle par défaut : environ 60 000 tokens pour répondre « OK ». Avec un prompt court et Haiku : environ 940 tokens. `total_cost_usd` est un équivalent au tarif API, non facturé avec l'abonnement. `apiKeySource = 'none'` ne prouve l'abonnement qu'avec une réponse réussie.

Outils intégrés : laisser `tools` par défaut et lister ceux autorisés sans confirmation, par ex. `allowed_tools=["Read", "Grep"]` . Nom d'un outil MCP : `mcp__<serveur>__<outil>` , joker `mcp__<serveur>__*` .

Agent IA : composants et usage en local

Suite : claude -p, Claude Desktop, clé API, Ollama, déclencheurs locaux

bstorm TRAINING

Mode A · Abonnement (suite) — sans code

`claude -p` : un run non interactif

```
# exécutable embarqué dans le SDK (ou « claude » si Claude Code installé)
$claude = ".\.venv\Lib\site-packages\claude_agent_sdk\_bundled\claude.exe"
& $claude -p "Résume notes.txt en 5 lignes" `
    --model claude-haiku-5-5 --allowedTools "Read"
Get-Content erreurs.log |
    & $claude -p "Liste les causes probables" --output-format json
& $claude -p "Ajoute une note 'test'" --mcp-config mcp.json `
    --strict-mcp-config --allowedTools "mcp__notes__*" `
    --permission-mode dontAsk
```

- `--output-format` : `text` (défaut), `json` (champ `result`, coût, `session_id`), `stream-json`.
- `--system-prompt` remplace le prompt système, `--append-system-prompt` le complète. `--max-budget-usd` plafonne la dépense.
- `--permission-mode dontAsk` : refuse tout ce qui demanderait une validation (runs sans surveillance).
- `--bare` ignore la connexion au compte et exige `ANTHROPIC_API_KEY` : ne pas l'utiliser en mode abonnement.
- Code retour 0 = succès, autre = échec : exploitable dans un script.

Claude Desktop + serveur MCP

Déclarer le serveur dans `%APPDATA%\Claude\claude_desktop_config.json` (Paramètres > Développeur > Modifier la config), quitter complètement l'application puis la relancer. Détail : fiche « MCP en pratique ».

Mode B · Clé API — SDK anthropic

```
.\.venv\Scripts\python.exe -m pip install anthropic
$env:ANTHROPIC_API_KEY = "sk-ant-..."  # clé créée sur platform.claude.com
```

Tool runner : la boucle fournie par le SDK

```
from anthropic import Anthropic, beta_tool
client = Anthropic()                        # lit ANTHROPIC_API_KEY

@beta_tool
def lire_fichier(chemin: str) -> str:
    """Lit un fichier texte local.

    Args:
        chemin: chemin du fichier
    """
    return open(chemin, encoding="utf-8").read()

runner = client.beta.messages.tool_runner(
    model="claude-haiku-5-5", max_tokens=1024, max_iterations=5,
    tools=[lire_fichier],
    messages=[{"role": "user", "content": "Résume notes.txt"}])
for message in runner:            # un message par tour
    print(message.content)
# ou directement : final = runner.until_done()
```

`@beta_tool` déduit le schéma JSON des annotations de type et de la docstring (`Args:`). Boucle manuelle (`client.messages.create` + traitement des blocs `tool_use`) pour valider chaque appel ou journaliser finement.

Managed Agents : boucle et bac à sable hébergés

```
agent = client.beta.agents.create(name="Assistant",
    model="claude-haiku-5-5", system="Tu es un assistant concis.",
    tools=[{"type": "agent_toolset_20260401"}])  # bash, fichiers, web
env = client.beta.environments.create(name="env-test",
    config={"type": "cloud",
            "networking": {"type": "limited",
                           "allow_package_managers": True}})
session = client.beta.sessions.create(agent=agent.id,
                                      environment_id=env.id)
texte = {"type": "text", "text": "Crée fibo.py et exécute-le"}
msg = {"type": "user.message", "content": [texte]}
with client.beta.sessions.events.stream(session.id) as flux:
    client.beta.sessions.events.send(session.id, events=[msg])
    for ev in flux:
        if ev.type == "agent.message":
            for b in ev.content:
                print(getattr(b, "text", ""), end="")
        elif ev.type == "session.status_idle":
            break
```

Agent (modèle, prompt, outils), environnement (bac à sable cloud ou auto-hébergé), session (un run), événements (flux). En-tête bêta `managed-agents-2026-04-01`. L'agent s'exécute hors du poste : pas d'accès direct aux fichiers locaux.

Mode C · Local gratuit — Ollama

```
# installer : OllamaSetup.exe (ollama.com/download) ou
#   irm https://ollama.com/install.ps1 | iex   (sans droits admin)
ollama pull qwen3            # modèle compatible appel d'outils
ollama run qwen3 "Bonjour"    # test direct
ollama list                  # modèles présents
# API locale : http://localhost:11434
.\.venv\Scripts\python.exe -m pip install ollama
```

```
from ollama import chat

def lire_fichier(chemin: str) -> str:
    """Lit un fichier texte local.

    Args:
        chemin: chemin du fichier
    """
    return open(chemin, encoding="utf-8").read()

outils = {"lire_fichier": lire_fichier}
messages = [{"role": "user", "content": "Résume notes.txt"}]
for _ in range(5):                          # garde-fou : 5 tours
    r = chat(model="qwen3", messages=messages,
             tools=list(outils.values()))
    messages.append(r.message)
    if not r.message.tool_calls:           # réponse finale
        print(r.message.content)
        break
    for appel in r.message.tool_calls:     # exécuter chaque appel
        nom = appel.function.name
        res = outils[nom](**appel.function.arguments)
        messages.append({"role": "tool", "tool_name": nom,
                         "content": str(res)})
```

Compatibilité API Anthropic (sous-ensemble) : `Anthropic(base_url="http://localhost:11434", api_key="ollama")`, avec un nom de modèle Ollama.

Déclencheurs locaux

Manuel : `.\.venv\Scripts\python.exe agent.py`, ou un raccourci vers un `.cmd`.

Boucle de surveillance (`surveiller.py`, laissé ouvert dans un terminal) :

```
import time
from pathlib import Path
ENTREE, FAIT = Path("entree"), Path("fait")
ENTREE.mkdir(exist_ok=True); FAIT.mkdir(exist_ok=True)
while True:
    for f in sorted(ENTREE.glob("*.txt")):
        traiter(f)                # appel de l'agent sur le fichier
        f.rename(FAIT / f.name)   # déplacer = ne pas retraiter
    time.sleep(5)
```

Planificateur de tâches Windows. Écrire `C:\agents\lancer.cmd` :

```
cd /d C:\agents
.venv\Scripts\python.exe agent.py >> agent.log 2>&1
```

```
# tous les jours à 8 h
schtasks /Create /SC DAILY /ST 08:00 /TN "agent-rapport" `
         /TR "C:\agents\lancer.cmd"
# toutes les 15 minutes
schtasks /Create /SC MINUTE /MO 15 /TN "agent-veille" `
         /TR "C:\agents\lancer.cmd"
schtasks /Run /TN "agent-rapport"       # lancer tout de suite
schtasks /Query /TN "agent-rapport" /V /FO LIST
schtasks /Delete /TN "agent-rapport" /F
```

Créer la tâche avec l'utilisateur Windows connecté au compte Claude : les identifiants d'abonnement sont dans son profil (`%USERPROFILE%\.claude\`). Le `.cmd` fixe le dossier de travail et capture la sortie.

cron (Linux, macOS, WSL) : `crontab -e`, puis

```
# du lundi au vendredi à 8 h
0 8 * * 1-5  cd $HOME/agents && .venv/bin/python agent.py >> log 2>&1
```

Routine locale Claude Desktop (mode A)

- Claude Desktop 1.1.5368 ou plus. Onglet **Code** > **Routines** > **New routine** > **Local**. Champs : nom, description, instructions (prompt, mode de permission, modèle), dossier de travail, planification.
- Fréquences : Manual, Hourly, Daily, Weekdays, Weekly ; autre fréquence en la demandant dans une session (« toutes les 6 heures »). Intervalle minimal : 1 minute.
- Ne s'exécute que si l'application est ouverte et le poste éveillé (réglage **Keep computer awake**). Run manqué : un seul rattrapage au réveil (sur 7 jours).
- Faire un premier **Run now** et répondre « toujours autoriser » aux demandes de permission, sinon les runs suivants restent bloqués.
- Prompt stocké dans `~/.claude/scheduled-tasks/<nom>/SKILL.md`.